LUME PARA WINDOWS · BIBLIOTECA DE PDF E DOCX

Lume

Banco de dados

Diagrama de relacionamentos, dicionário de dados, busca em texto completo com FTS5, integridade, consultas das Estatísticas, backup, manutenção e evolução do SQLite embutido no Java.

ESQUEMA	BANCO	VERSÃO DO ESQUEMA	EDIÇÃO
11 tabelas · 1 virtual (FTS5)	SQLite 3.53 · sqlite-jdbc	user_version 2	2.1.0 · outubro de 2026

Erik Pimenta e Clara Portes

Copyright © 2026 Erik Pimenta e Clara Portes · Licença MIT

SUMÁRIO

Conteúdo

01 · Visão rápida 03

02 · Visão geral 04

- POR QUE SQLITE 04
- COMO O BANCO É ABERTO 04
- ARQUIVOS NA PASTA DE DADOS 04

03 · Diagrama de relacionamentos 06

- CARDINALIDADES 07
- CONVENÇÕES E TIPOS 08

04 · Dicionário de dados 09

- DOCUMENTS 09
- DOCUMENT_CONTENT 10
- TAGS 11
- DOCUMENT_TAGS 11
- COLLECTIONS 12
- COLLECTION_ITEMS 12
- READING_STATES 12
- ANNOTATIONS 13
- JOBS 14
- SETTINGS 14

05 · Preferências e integrações 16

06 · Ciclo de vida e estados 18

07 · Busca em texto completo 20

- DA CAIXA DE BUSCA PARA O MATCH 20
- RANKING E TRECHO 20
- PARECIDOS E PERGUNTE 21

08 · Integridade e transações 22

- AÇÕES EM LOTE 22

09 · Consultas das Estatísticas e úteis 24

10 · Backup, restauração e manutenção 26

- BACKUP PELO LUME 26
- RESTAURAÇÃO E MUDANÇA DE COMPUTADOR 26
- MANUTENÇÃO 26

11 · Evolução do esquema e versão web 28

12 · Histórico de versões 29

CAPÍTULO 01

Visão rápida

O Lume para Windows guarda tudo num único arquivo SQLite, o `lume.db`, dentro da pasta de dados. Este manual descreve o esquema, as regras que o banco garante sozinho, a busca, as consultas por trás das telas novas e como fazer backup, verificar e evoluir o banco.

11 tabelas 10 comuns + 1 virtual FTS5	**92** colunas 33 só em documents	**8** chaves estrangeiras todas ON DELETE CASCADE	**20** índices 10 declarados · 10 de PK/UNIQUE
22 restrições CHECK domínios, faixas e json_valid	**2** migrações PRAGMA user_version = 2	**5** chaves em settings profile · ai · window · watch_folders · webhooks	**0** tabelas novas na 2.1.0 integrações em settings

Onde cada coisa mora

LUME.DB · FONTE DA VERDADE

- Documentos, metadados, sumário, descrição por IA e texto extraído.
- Índice de busca FTS5.
- Etiquetas, coleções, estantes, progresso e anotações.
- Histórico de cada etapa do pipeline.
- Preferências, chave de IA cifrada, pastas monitoradas e webhooks.

MEMÓRIA DO PROCESSO · TRANSPORTE

- Filas de leitura e descrição (`ExecutorService`).
- Novas tentativas agendadas (5 s, 15 s, 45 s).
- Eventos ao vivo (SSE) e envio de webhooks.
- Agendador das pastas monitoradas e a lista de arquivos já recusados.

DOCUMENTOS/ · ARQUIVOS

- Cópia do original, sem alteração.
- Capa em JPEG.
- HTML de leitura higienizado e imagens dos DOCX.
- Importações e backups em andamento em `tmp/`.

> REGRA DE OURO
>
> Fechar o Lume no meio de um trabalho não perde dados: o que estava em memória é refeito a partir do banco na próxima abertura. Perder o `lume.db` ou a pasta `documentos/` perde dados — por isso o backup da 2.1.0 leva os dois juntos (capítulo 10).

Para quem é este documento

Para quem usa, mantém ou desenvolve o Lume: a referência de cada tabela e coluna, as regras que o banco e a aplicação garantem e as tarefas de rotina (consultas, verificação, backup, restauração e migrações). Tipos, nulos e padrões das fichas foram lidos do próprio esquema (`Migrations.java`), que é a fonte da verdade.

CAPÍTULO 02

Visão geral

Um banco embutido, sem servidor, aberto por uma conexão só e protegido por restrições: é o suficiente para uma biblioteca pessoal e cabe num pendrive.

Por que SQLite

- **Zero instalação**: o SQLite vem dentro do próprio Lume, no driver JDBC `sqlite-jdbc` 3.53 (Xerial), que já traz o motor compilado com FTS5 e JSON. Não há servidor, porta nem senha.
- **Um usuário, um computador**: o Lume desktop não tem contas nem compartilhamento, então a concorrência é baixa e o SQLite sobra.
- **Portátil**: copiar a pasta de dados (ou um backup em zip) leva a biblioteca inteira para outro computador ou para um pendrive.
- **Busca embutida**: o FTS5 faz a busca em texto completo sem acentos, com ranking BM25 e trechos destacados.
- **Compatível**: o esquema é o mesmo desde a versão em Electron (esquema 2). A 2.1.0 não muda o esquema: as integrações ficam na tabela `settings`.

Como o banco é aberto

A classe `Database` (pacote `io.github.ravelroot.lume.db`) abre uma conexão JDBC só, `jdbc:sqlite:<pasta>/lume.db`, e liga o modo de trabalho:

```
PRAGMA journal_mode = WAL;      -- leituras não esperam escritas; grava lume.db-wal ao lado
PRAGMA synchronous = NORMAL;    -- seguro com WAL e bem mais rápido que FULL
PRAGMA foreign_keys = ON;       -- o SQLite só aplica chaves estrangeiras com isto ligado
PRAGMA busy_timeout = 5000;     -- espera até 5 s se o arquivo estiver ocupado
```

Em seguida `migrate()` compara `PRAGMA user_version` com a lista `Migrations.ALL` e aplica as que faltam, cada uma numa transação. Se o banco tiver uma versão maior que a do aplicativo (foi aberto por um Lume mais novo), a abertura é recusada com uma mensagem clara. Ao fechar, o Lume faz `PRAGMA wal_checkpoint(TRUNCATE)`, devolvendo tudo para o `lume.db`.

As consultas passam pelo `JdbcTemplate` do Spring, sobre um `SingleConnectionDataSource`. Uma trava (`ReentrantLock`) garante que a API, o pipeline, as pastas monitoradas, os webhooks e as ferramentas usem a conexão um de cada vez.

Arquivos na pasta de dados

CAMINHO	CONTEÚDO
`lume.db`	O banco (fonte da verdade de tudo que não é arquivo)
`lume.db-wal` · `lume.db-shm`	Diário do modo WAL; somem ou ficam vazios quando o Lume fecha
`documentos/<id>/original.pdf|docx`	Cópia do arquivo importado
`documentos/<id>/capa.jpg`	Capa (primeira página ou capa tipográfica)

CAMINHO	CONTEÚDO
`documentos/<id>/leitura.html` · `media/`	HTML de leitura higienizado e imagens dos DOCX
`tmp/`	Cópias em andamento durante a importação e o instantâneo do banco durante o backup
`lume.lock` · `instancia.json`	Trava de instância única e porta/chave do Lume aberto
`lume.log`	Registro do aplicativo (inclui as pastas monitoradas e os webhooks)

> **BANCO E ARQUIVOS ANDAM JUNTOS**
>
> O `id` do documento é o nome da pasta em `documentos/`. Ao excluir um documento (sozinho ou em lote), o Lume apaga a linha (e, em cascata, tudo que depende dela) e depois a pasta. Nunca apague pastas à mão com o Lume aberto; para limpar sobras, use **Ferramentas → Limpar sobras**.

CAPÍTULO 03

Diagrama de relacionamentos

As 11 tabelas giram em torno de `documents`. Todas as chaves estrangeiras usam `ON DELETE CASCADE`: apagar um documento apaga o texto, a leitura, as anotações, as etiquetas ligadas, os itens de coleção e o histórico de etapas.

O pé de galinha fica do lado “muitos” e o traço duplo do lado “um”. Cada caixa mostra as colunas principais; as fichas do capítulo 04 trazem todas. A faixa colorida indica a área: ACERVO ORGANIZAÇÃO LEITURA SISTEMA. A tabela virtual `documents_fts` não tem chave estrangeira: a ligação é mantida pela aplicação.



FIGURA 1 Diagrama de relacionamentos do `lume.db` (esquema 2, sem mudanças na 2.1.0).

Cardinalidades

RELAÇÃO	TIPO	LEITURA
`documents` → `document_content`	1 : 0..1	O texto é criado na leitura; documentos na fila ainda não têm.
`documents` → `reading_states`	1 : 0..1	Criado na primeira interação (abrir, estante, favorito).
`documents` → `annotations`	1 : N	Destaques, notas e marcadores.
`documents` → `jobs`	1 : N	Uma linha por etapa e tentativa.
`documents` ↔ `tags`	N : N	Por `document_tags`, com a origem (IA ou pessoa).
`documents` ↔ `collections`	N : N	Por `collection_items`, com a posição.
`documents` → `documents_fts`	1 : 1 lógico	Mantido por `SearchService.refreshSearch` na mesma transação.

RELAÇÃO	TIPO	LEITURA
`settings.watch_folders[].collection` → `collections.id`	N : 0..1 lógico	Validado pela aplicação; uma coleção apagada deixa a pasta sem destino.

Convenções e tipos

- **Datas**: texto ISO 8601 em UTC (`2026-10-27T13:00:00.315Z` , gerado por `Times.now()`), que ordena corretamente como texto e permite `substr(created_at, 1, 7)` para agrupar por mês.
- **Booleanos**: `INTEGER` 0 ou 1 com `CHECK` .
- **JSON**: campos estruturados (sumário, metadados, posição de leitura, âncora, detalhes da etapa, preferências) são texto validado com `CHECK (json_valid(...))` , lidos com o Jackson e consultados com `json_extract` .
- **Identificadores**: UUID v4 em `TEXT` para documentos, coleções e anotações; `INTEGER PRIMARY KEY` (rowid) para etiquetas e etapas.
- **Nomes**: tabelas no plural, colunas em `snake_case` , índices `<tabela>_<coluna>` .

CAPÍTULO 04

Dicionário de dados

Uma ficha por tabela, coluna por coluna, com as regras que o banco aplica e as que a aplicação garante.

[1] Chaves primárias de texto: por compatibilidade histórica, o SQLite aceitaria `NULL` nelas; a aplicação sempre grava um valor (UUID, chave de preferência ou o id do documento). Índices automáticos de `PRIMARY KEY` e `UNIQUE` aparecem com esses nomes.

FICHA 01 ACERVO — 33 colunas · 4 índices

documents

Um arquivo importado: identidade, metadados, capa, sumário, descrição por IA e estado no pipeline. É o centro do esquema; tudo o mais aponta para ele e é apagado junto.

Referenciada por `document_content`, `document_tags`, `collection_items`, `reading_states`, `annotations` e `jobs` (todas com `ON DELETE CASCADE`).

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	TEXT	não[1]	—	UUID v4 gerado na importação; também é o nome da pasta em `documentos/`.
title	TEXT	não	—	Título (até 300). Vem dos metadados bons do arquivo ou do nome do arquivo.
title_sort	TEXT	não	''	Título minúsculo e sem acentos: ordenação e busca por trecho do título (`LIKE`).
author	TEXT	não	''	Autor (até 200).
author_sort	TEXT	não	''	Autor normalizado, para ordenar.
description	TEXT	não	''	Descrição escrita pela pessoa (até 5000).
metadata_locked	INTEGER	não	0	1 depois que a pessoa edita título ou autor: a releitura não sobrescreve.
summary	TEXT	não	''	Descrição por IA (2 a 3 frases).
summary_source	TEXT	não	''	Quem escreveu: `local`, `anthropic` ou `openai`.
summary_model	TEXT	não	''	Modelo usado, quando externo.
summary_at	TEXT	sim	—	Quando a descrição foi gravada.
kind	TEXT	não	—	Tipo decidido pelo conteúdo, não pela extensão.
original_name	TEXT	não	—	Nome do arquivo importado.
source_path	TEXT	não	''	Caminho de origem (só informativo); as pastas monitoradas o usam para não importar de novo.
size_bytes	INTEGER	não	—	Tamanho do original (até 500 MB por regra da aplicação).
sha256 UK	TEXT	não	—	Impressão digital do conteúdo: o mesmo arquivo não entra duas vezes.
page_count	INTEGER	sim	—	Páginas (no DOCX, do `app.xml` ou estimadas).
word_count	INTEGER	sim	—	Palavras do texto extraído.
language	TEXT	não	''	Idioma estimado (`pt`, `en`…); alimenta as Estatísticas.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
cover_width	INTEGER	sim	—	Largura da capa.
cover_height	INTEGER	sim	—	Altura da capa.
cover_color	TEXT	não	`''`	Cor média da capa (ou o tom da paleta).
cover_generated	INTEGER	não	`0`	1 se é a capa tipográfica.
cover_version	INTEGER	não	`0`	Sobe a cada capa nova; vai na URL para furar o cache.
has_reader_html	INTEGER	não	`0`	1 se existe `leitura.html` (DOCX).
outline	TEXT	não	`'[]'`	Sumário: `[{title, page, level, anchor}]`, até 500 itens.
metadata	TEXT	não	`'{}'`	Assunto, palavras-chave, datas, programa de origem, avisos da leitura.
status	TEXT	não	`'queued'`	Etapa do pipeline (capítulo 06).
status_detail	TEXT	não	`''`	Mensagem para a pessoa (erro ou nova tentativa).
attempts	INTEGER	não	`0`	Tentativas da etapa atual.
created_at	TEXT	não	—	Importação; base de “Documentos adicionados” nas Estatísticas.
updated_at	TEXT	não	—	Última mudança.
processed_at	TEXT	sim	—	Primeira vez que ficou pronto.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
documents_created	(created_at DESC)	Ordem padrão da biblioteca.
documents_status	(status)	Filtros, recuperação e manutenção.
documents_title_sort	(title_sort)	Ordem por título e busca por trecho.
UNIQUE	(sha256)	Um arquivo, um documento.

VERIFICAÇÕES `kind IN ('pdf','docx')` · `size_bytes > 0` · `status IN ('queued','extracting','describing','ready','failed')` · booleanos em 0/1 · `json_valid(outline)` e `json_valid(metadata)`.

FICHA 02 ACERVO — 4 colunas · 0 índices

document_content

O texto extraído de um documento, separado para as listagens ficarem leves. É a matéria-prima da busca, do “Pergunte ao documento” e da descrição.

Aponta para `document_id → documents.id` (1:1).

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
document_id PK FK	TEXT	não[1]	—	O documento (um texto por documento).
text	TEXT	não	`''`	Texto limpo, até 2.000.000 de caracteres; no PDF as páginas são separadas por `\f`, o que dá a página de cada trecho do “Pergunte”.
text_truncated	INTEGER	não	`0`	1 se o texto passou do limite e foi cortado.
updated_at	TEXT	não	—	Última leitura.

FICHA 03 ORGANIZAÇÃO

4 colunas · 1 índice

tags

Uma etiqueta da biblioteca. O `slug` sem acentos garante uma etiqueta por ideia (programação e programacao são a mesma).

Referenciada por `document_tags.tag_id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	INTEGER	não	—	Identificador interno (rowid).
name	TEXT	não	—	Nome em minúsculas (até 40).
slug UK	TEXT	não	—	Forma sem acentos; filtro `?tag=` da biblioteca.
created_at	TEXT	não	—	Criação.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
UNIQUE	(slug)	Uma etiqueta por ideia.

VERIFICAÇÕES Etiquetas sem nenhum documento são apagadas na mesma transação que as deixou órfãs.

FICHA 04 ORGANIZAÇÃO

4 colunas · 2 índices

document_tags

Liga documentos e etiquetas (N:N) e guarda quem sugeriu: a IA ou a pessoa. A ação em lote “Adicionar etiquetas” grava com origem `usuario` sem apagar as anteriores.

Aponta para `document_id → documents.id` e `tag_id → tags.id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
document_id PK FK	TEXT	não	—	O documento.
tag_id PK FK	INTEGER	não	—	A etiqueta.
source	TEXT	não	`'usuario'`	`ia` ou `usuario`: a IA só troca as dela; a edição manual substitui todas.
created_at	TEXT	não	—	Quando a ligação foi feita.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
PRIMARY KEY	(document_id, tag_id)	A mesma etiqueta uma vez por documento.
document_tags_tag	(tag_id)	Filtro por etiqueta, contagens e “Parecidos”.

VERIFICAÇÕES `source IN ('ia','usuario')`. Até 12 etiquetas por documento (aplicação).

FICHA 05 ORGANIZAÇÃO — 5 colunas · 1 índice

collections

Uma coleção criada pela pessoa. Pode ser o destino de uma pasta monitorada.

Referenciada por `collection_items.collection_id` (e, por id, por `settings.watch_folders`).

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	TEXT	não[1]	—	UUID.
name	TEXT	não	—	Nome (único sem diferenciar maiúsculas).
description	TEXT	não	`''`	Até 300 caracteres.
created_at	TEXT	não	—	Criação.
updated_at	TEXT	não	—	Última mudança.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
collections_name	UNIQUE (lower(name))	Nomes de coleção únicos.

VERIFICAÇÕES `length(trim(name)) > 0`.

FICHA 06 ORGANIZAÇÃO — 4 colunas · 2 índices

collection_items

Um documento dentro de uma coleção, com a posição. Um documento pode estar em várias coleções.

Aponta para `collection_id → collections.id` e `document_id → documents.id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
collection_id PK FK	TEXT	não	—	A coleção.
document_id PK FK	TEXT	não	—	O documento.
position	INTEGER	não	0	Ordem na coleção (1, 2, 3…); novos entram no fim.
added_at	TEXT	não	—	Desempate da ordem.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
PRIMARY KEY	(collection_id, document_id)	Um documento uma vez por coleção.
collection_items_document	(document_id)	Coleções de um documento; exportações.

FICHA 07 LEITURA — 10 colunas · 1 índice

reading_states

O estado de leitura de um documento: estante, página, progresso, favorito e datas. Criado na primeira interação; alimenta o “Continuar lendo”, as Estantes e as Estatísticas.

Aponta para `document_id → documents.id` (1:1).

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
document_id PK FK	TEXT	não[1]	—	Um estado por documento.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
shelf	TEXT	não	`''`	`''`, `para_ler`, `lendo` ou `lido`.
page	INTEGER	não	`1`	Página atual (limitada ao total pela aplicação).
progress	REAL	não	`0`	Fração lida, de 0 a 1.
location	TEXT	não	`'{}'`	Posição no DOCX: `{"block": 42}`.
favorite	INTEGER	não	`0`	Favorito.
started_at	TEXT	sim	—	Primeira vez em “Lendo”.
finished_at	TEXT	sim	—	Quando foi para “Lido” (zera ao sair); base de “Leituras concluídas”.
last_opened_at	TEXT	sim	—	Última abertura no leitor.
updated_at	TEXT	não	—	Última mudança.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
reading_states_recent	(last_opened_at DESC)	“Continuar lendo”.

VERIFICAÇÕES `shelf IN ('','para_ler','lendo','lido')` · `page >= 1` · `progress BETWEEN 0 AND 1` · `json_valid(location)`.

FICHA 08 LEITURA — 10 colunas · 1 índice

annotations

Um destaque, nota ou marcador. Exportadas em Markdown por documento e nas notas para Obsidian; cada criação emite `annotation.created`.

Aponta para `document_id → documents.id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	TEXT	não[1]	—	UUID.
document_id FK	TEXT	não	—	O documento.
kind	TEXT	não	—	`note`, `highlight` ou `bookmark`; nota exige texto.
page	INTEGER	sim	—	Página (PDF) ou nulo (DOCX).
anchor	TEXT	não	`'{}'`	PDF: `{"rects": [[x, y, w, h]]}` em frações da página; DOCX: `{"block": n}`.
quote	TEXT	não	`''`	Trecho destacado (até 4000).
note	TEXT	não	`''`	Texto da nota (até 8000).
color	TEXT	não	`'salvia'`	Cor de destaque da paleta.
created_at	TEXT	não	—	Criação.
updated_at	TEXT	não	—	Última edição.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
annotations_document	(document_id, page)	Anotações de um documento em ordem de página.

VERIFICAÇÕES `kind IN ('note','highlight','bookmark')` · `color IN ('salvia','rose','ardosia','camurca')` · `json_valid(anchor)`. Até 2000 por documento (aplicação).

FICHA 09 SISTEMA

10 colunas · 2 índices

jobs

Uma etapa do pipeline (leitura ou descrição) de um documento: o histórico da tela Sistema.

Aponta para `document_id → documents.id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	INTEGER	não	—	Identificador (rowid).
document_id FK	TEXT	não	—	O documento.
stage	TEXT	não	—	`extract` ou `describe`.
status	TEXT	não	`'running'`	`running`, `succeeded`, `retrying` ou `failed`.
attempt	INTEGER	não	`1`	Número da tentativa.
started_at	TEXT	não	—	Início.
finished_at	TEXT	sim	—	Fim.
duration_ms	INTEGER	sim	—	Duração (médias da tela Sistema).
error	TEXT	não	`''`	Motivo da falha ou aviso.
details	TEXT	não	`'{}'`	Páginas, palavras, provedor, modelo e se caiu para o resumo local (`fallback`).

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
jobs_started	(started_at DESC)	Últimas etapas.
jobs_document	(document_id)	Histórico de um documento.

VERIFICAÇÕES `stage IN ('extract','describe')` · `status IN (…)` · `json_valid(details)`.

FICHA 10 SISTEMA

3 colunas · 0 índices

settings

Preferências guardadas como JSON por chave: perfil, IA, janela e, desde a 2.1.0, as pastas monitoradas e os webhooks (capítulo 05).

Sem chaves estrangeiras.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
key PK	TEXT	não[1]	—	Nome da preferência.
value	TEXT	não	—	Valor em JSON.
updated_at	TEXT	não	—	Última gravação.

VERIFICAÇÕES `json_valid(value)`.

FICHA 11 ACERVO

5 colunas · tabela virtual

documents_fts

O índice de busca em texto completo (FTS5): uma linha por documento, reconstruída sempre que algo pesquisável muda. Detalhes no capítulo 07.

Ligação lógica com `documents.id` por `document_id` (sem chave estrangeira; tabelas virtuais não têm).

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
document_id	UNINDEXED	—	—	Liga a linha ao documento; não é pesquisado.
title	texto	—	—	Título (peso 10 no ranking).
author	texto	—	—	Autor (peso 5).
extra	texto	—	—	Descrição por IA, descrição da pessoa e etiquetas (peso 3).
body	texto	—	—	Os primeiros 300.000 caracteres do texto (peso 1).

TOKENIZADOR `unicode61 remove_diacritics 2`: “reunião”, “reuniao” e “REUNIÃO” são a mesma palavra.

CAPÍTULO 05

Preferências e integrações

A tabela `settings` guarda um JSON por chave. Na 2.1.0 ela ganhou duas chaves, `watch_folders` e `webhooks`, em vez de tabelas novas: são poucas linhas, lidas inteiras, e assim o esquema continua o mesmo.

CHAVE	QUEM GRAVA	VALOR
`profile`	`SettingsService`	`{name, theme, reader_prefs: {theme, fontSize}}`
`ai`	`SettingsService`	`{provider, model, base_url, api_key, timeout_s, max_input_chars}`; `api_key` é `dpapi:` + Base64 do texto cifrado pelo cofre do Windows
`window`	`DesktopController`	`{x, y, width, height, maximized}` da janela
`watch_folders` 2.1.0	`WatchFolderService`	Lista de até 10 pastas monitoradas
`webhooks` 2.1.0	`WebhookService`	Lista de até 10 webhooks; o segredo é cifrado como a chave de IA

watch_folders uma pasta por item

```
[{
  "id": "0b6f2c9e-…",                      // UUID
  "path": "C:\\Users\\voce\\Downloads",       // pasta absoluta
  "collection": "5d1a…" ,                  // coleção de destino ou null
  "enabled": true,                         // false = pausada
  "last_scan_at": "2026-10-27T13:02:10.512Z",
  "imported_total": 14,                    // quantos entraram por esta pasta
  "created_at": "2026-10-27T12:40:01.004Z"
}]
```

webhooks um endereço por item

```
[{
  "id": "a3c9…",
  "url": "https://hooks.slack.com/services/…",
  "format": "slack",                       // json · slack · discord
  "events": ["document.ready", "shelf.finished"],
  "enabled": true,
  "secret": "dpapi:AQAAANCMnd8B…",           // "" quando não há segredo
  "last_status": 200, "last_error": null, "last_at": "2026-10-27T13:05:44.120Z",
  "created_at": "2026-10-27T12:41:30.870Z"
}]
```

As duas listas são lidas e regravadas inteiras a cada mudança (`INSERT … ON CONFLICT (key) DO UPDATE`), dentro da trava da conexão. A API nunca devolve o segredo: `GET /api/v1/integrations` mostra só `has_secret`. O que não é guardado de propósito: a lista de arquivos já recusados pelas pastas monitoradas (fica em memória e recomeça a cada abertura) e o histórico de envios dos webhooks (só o último resultado fica salvo).

Consultar à mão

```
SELECT json_extract(value, '$[0].path'), json_array_length(value) FROM settings WHERE key =
'watch_folders';
SELECT json_extract(h.value, '$.url'), json_extract(h.value, '$.last_status')
  FROM settings s, json_each(s.value) h WHERE s.key = 'webhooks';
```

CAPÍTULO 06

Ciclo de vida e estados

Três colunas funcionam como pequenas máquinas de estado: o estado do documento, o de cada etapa e a estante. Quem muda o estado do documento é sempre o `PipelineService`, numa transação, e cada mudança vira um evento para a tela.

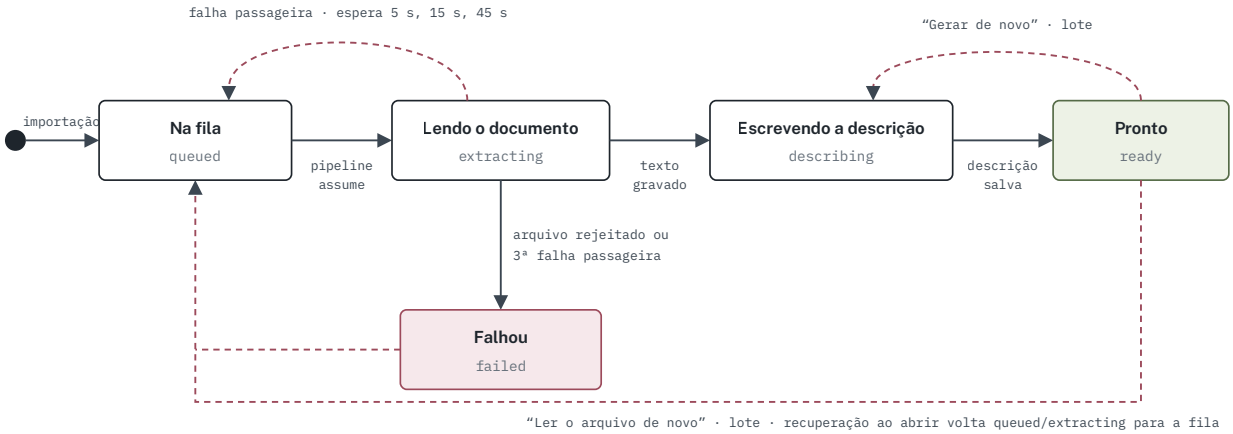


FIGURA 2 Estados do documento (`documents.status`). Ao chegar em `ready`, o pipeline emite `document.ready`.

TRANSIÇÃO	ONDE	O QUE É GRAVADO
importação → `queued`	`LibraryService.importFile`	Linha nova, arquivo em `documentos/<id>`, entrada no índice de busca; evento `document.created`
`queued` → `extracting`	`PipelineService.claim`	`attempts + 1`; só pega quem está no estado esperado (pedido repetido não faz nada)
`extracting` → `describing`	`applyExtraction`	Páginas, palavras, sumário, metadados, capa (`cover_version + 1`), texto em `document_content`, índice de busca
`describing` → `ready`	`runDescribe`	Descrição, origem, modelo, idioma, etiquetas da IA, `processed_at` na primeira vez; evento `document.ready`
falha passageira → `queued`	`runExtract`	`status_detail` "Nova tentativa em 5 s"; etapa `retrying`
falha definitiva → `failed`	`fail`	`status_detail` com a mensagem; etapa `failed`
lote `reprocess` · `redescribe`	`BatchService` · `ToolsService`	Volta para a fila de leitura ou de descrição (inclusive "Descrever tudo de novo")

> RECUPERAÇÃO AO ABRIR
>
> Quando o Spring termina de subir (`ApplicationReadyEvent`), `recover()` marca como `failed` as etapas que ficaram `running`, devolve para a fila de leitura os documentos em `queued` ou `extracting` e para a fila de descrição os que estavam em `describing`.

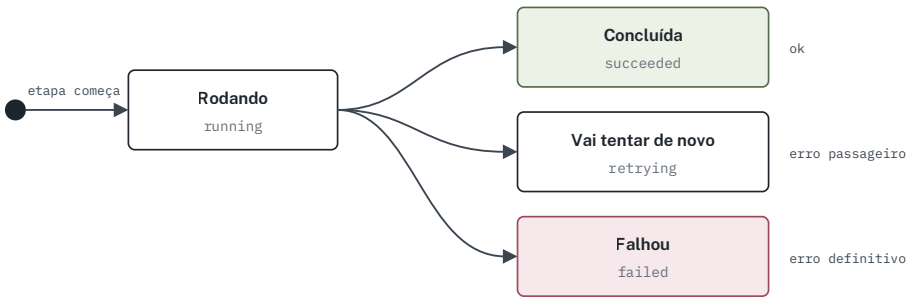


FIGURA 3 Estados de uma etapa (`jobs.status`).

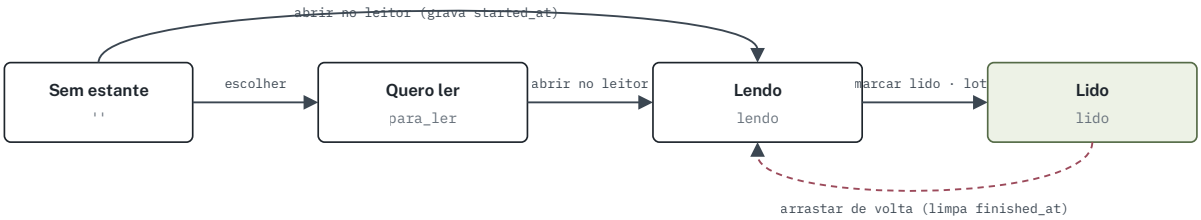


FIGURA 4 Estantes de leitura (`reading_states.shelf`). Chegar em `lido` grava `finished_at` e emite `shelf.finished`.

CAPÍTULO 07

Busca em texto completo

A busca usa a tabela virtual FTS5 com uma linha por documento. A mesma tabela serve à paleta de comandos e ao “Parecidos na sua biblioteca”; o “Pergunte ao documento” lê o texto em `document_content`.

```
CREATE VIRTUAL TABLE documents_fts USING fts5 (
  document_id UNINDEXED,     -- liga a linha ao documento (não é pesquisado)
  title,                     -- peso 10
  author,                    -- peso 5
  extra,                     -- descrição por IA + sua descrição + etiquetas, peso 3
  body,                      -- os primeiros 300.000 caracteres do texto, peso 1
  tokenize = 'unicode61 remove_diacritics 2'
);
```

`SearchService.refreshSearch(id)` apaga e reinsere a linha do documento; ele roda na importação, depois da leitura, depois da descrição, ao editar título, autor ou descrição e ao trocar etiquetas (inclusive em lote), sempre dentro da mesma transação da mudança. **Ferramentas → Reconstruir o índice de busca** faz isso para todos os documentos e compacta o índice.

Da caixa de busca para o MATCH

VOCÊ DIGITA	VIRA	POR QUÊ
`contrato aluguel`	`"contrato"* AND "aluguel"*`	todas as palavras, aceitando o começo
`"reunião de planejamento"`	`"reunião de planejamento"`	frase exata
`python -flask`	`"python"* NOT "flask"*`	exclusão
`a"b OR`	`"a" AND "b" AND "or"*`	aspas e operadores digitados viram texto: a consulta nunca quebra

Além do `MATCH`, a listagem aceita documentos cujo `title_sort` contenha o texto digitado (`LIKE`), para achar pedaços no meio de palavras do título. O filtro de etiqueta junta `document_tags` e `tags` pelo `slug`.

Ranking e trecho

```
SELECT d.*
FROM documents d
LEFT JOIN (SELECT document_id, bm25(documents_fts, 0, 10.0, 5.0, 3.0, 1.0) AS rank
           FROM documents_fts WHERE documents_fts MATCH :consulta) f ON f.document_id = d.id
WHERE (f.document_id IS NOT NULL OR d.title_sort LIKE :trecho ESCAPE '\')
ORDER BY (f.rank IS NULL), f.rank, d.created_at DESC
LIMIT :tamanho OFFSET :inicio;

-- trecho com a palavra destacada, só para os documentos da página
SELECT document_id, snippet(documents_fts, 4, char(2), char(3), ' … ', 26)
FROM documents_fts WHERE documents_fts MATCH :consulta AND document_id IN (...);
```

O BM25 do FTS5 devolve números menores para os melhores resultados; os pesos fazem o título valer mais que o texto. O trecho usa os caracteres de controle 2 e 3 como marcadores: o texto é escapado para HTML e só então eles viram `<mark>`, então nada do documento vira HTML na tela.

Parecidos e Pergunte 2.1.0

Parecidos na sua biblioteca (`DiscoveryService`) soma duas pistas: etiquetas em comum (cada etiqueta compartilhada vale 3 pontos, ou 4 quando, no outro documento, ela foi escolhida pela pessoa) e uma busca FTS com os termos do título e das etiquetas do documento ligados por `OR` (o 1º resultado vale 2,5 pontos e o valor cai até o 30º). Só entram documentos prontos, sem o próprio, em ordem de pontuação.

```
-- etiquetas em comum com o documento :id
SELECT other.document_id AS id,
       sum(CASE WHEN other.source = 'usuario' THEN 4.0 ELSE 3.0 END) AS s
FROM document_tags mine
JOIN document_tags other ON other.tag_id = mine.tag_id AND other.document_id != mine.document_id
WHERE mine.document_id = :id
GROUP BY other.document_id;

-- termos do título e das etiquetas no índice ("termo"* a partir de 5 letras)
SELECT document_id FROM documents_fts
WHERE documents_fts MATCH '"python"* OR "guia" OR "programacao"*' AND document_id != :id
ORDER BY bm25(documents_fts, 0, 10.0, 5.0, 3.0, 1.0) LIMIT 30;
```

O **Pergunte ao documento** não usa o FTS: lê `document_content.text` do documento, divide em trechos de 400 a 700 caracteres (a página vem da contagem de `\f` no PDF) e pontua os trechos com um BM25 próprio em memória (`PassageFinder`). Nada é gravado: perguntas e respostas não ficam no banco.

CAPÍTULO 08

Integridade e transações

As regras importantes ficam no próprio banco, não só no código: mesmo um script que escreva direto no `lume.db` não consegue gravar um estado inválido.

REGRA	COMO O BANCO GARANTE
Um arquivo, um documento	`sha256 UNIQUE` em `documents`; duas importações simultâneas do mesmo arquivo (por exemplo, a pasta monitorada e um arrastar): a segunda vira "duplicado"
Valores dentro do domínio	`CHECK` em `kind`, `status`, `shelf`, `color`, `stage`, booleanos, `progress`, `page`, `size_bytes`
JSON válido	`CHECK (json_valid(...))` em `outline`, `metadata`, `location`, `anchor`, `details` e `value` (inclusive as listas de integrações)
Sem órfãos	`FOREIGN KEY ... ON DELETE CASCADE` em todas as tabelas filhas, com `PRAGMA foreign_keys = ON`
Nomes de coleção únicos	`UNIQUE INDEX ON collections (lower(name))`
Uma ligação por par	Chaves primárias compostas em `document_tags` e `collection_items`

Transações

`Database.transaction(...)` recebe um `Runnable` ou `Supplier`: a primeira chamada abre `BEGIN IMMEDIATE` (reserva a escrita na hora, evitando conflito no meio) e chamadas aninhadas viram `SAVEPOINT`. Se qualquer coisa lançar exceção, tudo é desfeito (`ROLLBACK` ou `ROLLBACK TO`).

```
db.transaction(() -> {
    db.update("INSERT INTO documents (...) VALUES (...)", ...);
    Files.move(tmp, pasta.resolve("original." + kind));      // falhou? a linha também volta
    search.refreshSearch(id);                                // SAVEPOINT aninhado
});
```

Ações em lote 2.1.0

`POST /api/v1/documents/batch` (`BatchService`) recebe até 500 ids, descarta repetidos e os que não existem e aplica a ação documento por documento, com os mesmos serviços das ações individuais: `ReadingService` para estante e favorito, `LibraryService.setTags` para etiquetas (somando às da pessoa, sem tocar nas da IA), `CollectionService` para coleções, `deleteDocument` para exclusão e o pipeline para reler ou redescrever. Cada documento é gravado pelas transações do próprio serviço (não há uma transação única para o lote): o que já foi aplicado continua valendo se um item falhar, e os eventos e webhooks saem como sempre.

> CONCORRÊNCIA
> Só o processo do Lume escreve no banco, por uma conexão só, protegida por uma trava (`ReentrantLock`). Os fios da API (Tomcat), do pipeline, das pastas monitoradas, dos webhooks e das ferramentas esperam a vez, então não há disputa, e o modo WAL deixa leituras externas correrem enquanto o pipeline grava. Operações longas, como `VACUUM`, seguram a trava só pelo tempo da instrução.

CAPÍTULO 09

Consultas das Estatísticas e úteis

A tela Estatísticas (`InsightsService` , `GET /api/v1/insights`) é feita só de consultas de agregação sobre o esquema existente. Para abrir o banco fora do Lume, feche o aplicativo e use o `sqlite3` ou o DB Browser for SQLite, sempre numa cópia.

Totais da biblioteca e das anotações

```
SELECT count(*) AS documents, coalesce(sum(page_count), 0) AS pages, coalesce(sum(word_count), 0)
AS words,
       coalesce(sum(size_bytes), 0) AS bytes, coalesce(sum(kind = 'pdf'), 0) AS pdf,
coalesce(sum(kind = 'docx'), 0) AS docx
FROM documents;

SELECT count(*) AS annotations, coalesce(sum(kind = 'highlight'), 0) AS highlights,
       coalesce(sum(kind = 'note'), 0) AS notes, coalesce(sum(kind = 'bookmark'), 0) AS bookmarks
FROM annotations;
```

Estantes, lidos no ano e favoritos

```
SELECT coalesce(sum(shelf = 'lido'), 0) AS finished,
       coalesce(sum(shelf = 'lido' AND finished_at >= '2026-01-01'), 0) AS this_year,
       coalesce(sum(shelf = 'lendo'), 0) AS reading, coalesce(sum(shelf = 'para_ler'), 0) AS
wanted,
       coalesce(sum(favorite = 1), 0) AS favorites
FROM reading_states;
```

Páginas lidas (aproximação)

```
SELECT coalesce(sum(CASE WHEN rs.shelf = 'lido' THEN coalesce(d.page_count, 0)
                         WHEN rs.last_opened_at IS NULL THEN 0
                         WHEN d.kind = 'pdf' THEN rs.page
                         ELSE CAST(round(rs.progress * coalesce(d.page_count, 0)) AS INTEGER)
END), 0)
FROM reading_states rs JOIN documents d ON d.id = rs.document_id;
```

Por mês (últimos 12 meses; os meses sem nada são completados com zero na aplicação)

```
SELECT substr(created_at, 1, 7) AS month, count(*) AS n
FROM documents WHERE created_at IS NOT NULL GROUP BY month;                          --
adicionados

SELECT substr(finished_at, 1, 7) AS month, count(*) AS n
FROM reading_states WHERE finished_at IS NOT NULL AND shelf = 'lido' GROUP BY month;  --
concluídos
```

Assuntos, maiores documentos e idiomas

```
SELECT t.name, t.slug, count(dt.document_id) AS count
FROM tags t JOIN document_tags dt ON dt.tag_id = t.id
GROUP BY t.id ORDER BY count DESC, t.name LIMIT 10;

SELECT id, title, kind, page_count, size_bytes FROM documents
ORDER BY coalesce(page_count, 0) DESC, size_bytes DESC LIMIT 5;

SELECT CASE WHEN language = '' THEN 'indefinido' ELSE language END AS language, count(*) AS count
FROM documents GROUP BY 1 ORDER BY count DESC;
```

Outras consultas úteis

```
-- Quantos documentos em cada estado
SELECT status, count(*) FROM documents GROUP BY status;

-- O que estou lendo, do mais recente para o mais antigo
SELECT d.title, r.page, d.page_count, round(r.progress * 100) AS pct, r.last_opened_at
FROM reading_states r JOIN documents d ON d.id = r.document_id
WHERE r.shelf = 'lendo' ORDER BY r.last_opened_at DESC;

-- Documentos que vieram de uma pasta monitorada (pelo caminho de origem)
SELECT title, source_path FROM documents WHERE source_path LIKE 'C:\Users\voce\Downloads\%'
ESCAPE '|';

-- Tempo médio de leitura e descrição nos últimos 7 dias
SELECT stage, count(*), round(avg(duration_ms)) AS ms FROM jobs
WHERE status = 'succeeded' AND started_at >= strftime('%Y-%m-%dT%H:%M:%fZ', 'now', '-7 days')
GROUP BY stage;

-- Documentos que caíram para o resumo local
SELECT d.title, j.error FROM jobs j JOIN documents d ON d.id = j.document_id
WHERE j.stage = 'describe' AND json_extract(j.details, '$.fallback') = 1;

-- Busca direto no índice
SELECT d.title FROM documents_fts f JOIN documents d ON d.id = f.document_id
WHERE documents_fts MATCH '"cooperativa"*' ORDER BY bm25(documents_fts, 0, 10, 5, 3, 1);
```

CAPÍTULO 10

Backup, restauração e manutenção

A biblioteca é o banco mais a pasta `documentos/`. Na 2.1.0, as duas coisas saem juntas num `.zip` pela tela Ferramentas, e as tarefas de manutenção que antes exigiam o `sqlite3` viraram botões.

Backup pelo Lume `2.1.0`

01 **Instantâneo consistente**: `ToolsService.backupTo` roda `VACUUM INTO 'tmp/…/lume.db'`, que grava uma cópia compacta e coerente do banco sem fechar a conexão e sem parar o Lume.

02 **Zip**: o arquivo escolhido no diálogo recebe `lume.db` (o instantâneo), `documentos/**` (só as pastas de documentos que existem no banco) e `LEIAME.txt` com a data, a versão, a contagem e as instruções. O zip é gravado primeiro com outro nome e só renomeado no fim: um backup interrompido não deixa um arquivo pela metade.

03 **Limpeza**: o instantâneo temporário é apagado.

Sem o Lume aberto, o jeito manual continua valendo: com o aplicativo fechado (o diário WAL é incorporado ao `lume.db`), copie a pasta de dados inteira.

Restauração e mudança de computador

01 Feche o Lume.

02 Abra a pasta de dados (`%APPDATA%\Lume\Biblioteca`, `Lume-dados\Biblioteca` no modo portátil ou `LUME_DATA_DIR`).

03 Guarde (ou apague) `lume.db`, `lume.db-wal`, `lume.db-shm` e `documentos`.

04 Extraia o zip ali dentro e abra o Lume. Se o backup veio de uma versão mais antiga, as migrações que faltam rodam na abertura.

> SEGREDOS NÃO VIAJAM
>
> A chave de API (`settings.ai.api_key`) e os segredos dos webhooks (`settings.webhooks[].secret`) são cifrados pelo DPAPI para a conta do Windows que os gravou. Em outro computador ou outra conta eles não abrem: informe de novo em Preferências e em Integrações. As pastas monitoradas restauradas apontam para caminhos do computador antigo; confira-as em Integrações.

Manutenção

BOTÃO EM FERRAMENTAS	O QUE RODA	QUANDO USAR
Verificar a biblioteca	`PRAGMA integrity_check;` + conferência de `original.*` de cada documento + pastas sem linha em `documents` + arquivos em `tmp/`	Depois de uma queda de energia ou se algo parecer estranho: a resposta deve ser `ok`
Reconstruir o índice de busca	`refreshSearch(id)` para cada documento e `INSERT INTO documents_fts(documents_fts) VALUES ('optimize');`	Resultados faltando na busca; depois de editar o banco à mão

BOTÃO EM FERRAMENTAS	O QUE RODA	QUANDO USAR
Otimizar o banco de dados	`PRAGMA wal_checkpoint(TRUNCATE); VACUUM; PRAGMA optimize;`	Depois de apagar muitos documentos; o resultado mostra o tamanho antes e depois
Limpar sobras	Apaga pastas de `documentos/` sem linha no banco e temporários de `tmp/`	Depois de quedas no meio de importações ou exclusões
Descrever tudo de novo	Põe na fila de descrição todos os documentos em `ready`	Depois de trocar o provedor de IA

Comandos que continuam só no `sqlite3`, com o Lume fechado:

```
PRAGMA foreign_key_check;                                    -- ninguém deixou órfãos editando à mão?
DELETE FROM jobs WHERE started_at < date('now', '-180 days');  -- limpar histórico antigo de etapas
.backup copia.db                                             -- cópia do banco pelo próprio sqlite3
```

CAPÍTULO 11

Evolução do esquema e versão web

Como mudar o banco sem perder dados e como ele se compara ao PostgreSQL da versão web.

Criar uma migração

01 Acrescente um texto SQL no fim da lista `ALL` em `src/main/java/io/github/ravelroot/lume/db/Migrations.java`. Nunca edite migrações já publicadas.

02 Prefira mudanças aditivas: `ALTER TABLE ... ADD COLUMN` com `DEFAULT`, tabelas e índices novos. Dados pequenos de configuração podem ir para `settings`, como as integrações da 2.1.0, sem migração nenhuma.

03 Para mudar o tipo ou as restrições de uma coluna, siga o roteiro do SQLite: criar a tabela nova, copiar com `INSERT ... SELECT`, apagar a antiga e renomear, tudo na mesma migração (a migração 2 faz exatamente isso com as cores das anotações).

04 Rode `mvn test`: o `DatabaseTest` abre um banco vazio, confere as tabelas, abre de novo para garantir que nada é reaplicado e testa a migração de um banco no esquema 1.

```
// src/main/java/io/github/ravelroot/lume/db/Migrations.java
static final String V3 = """
        ALTER TABLE reading_states ADD COLUMN seconds_read INTEGER NOT NULL DEFAULT 0;
        """;    // 3: exemplo — tempo total de leitura
public static final List<String> ALL = List.of(V1, V2, V3);
```

Comparação com a versão web

VERSÃO WEB (POSTGRESQL)	LUME PARA WINDOWS (SQLITE)
`accounts_user`, `accounts_apitoken`	Não existem: um usuário só; o perfil fica em `settings`
`library_document.owner_id` e `visibility`	Não existem: tudo é de quem usa o computador
`uuid` nativo · `jsonb`	UUID em `TEXT` · `TEXT` com `CHECK (json_valid(...))` e `json_extract`
`tsvector` com pesos A/B/C, `pt_unaccent`, `ts_headline`	FTS5 com `remove_diacritics 2`, pesos no `bm25()` e `snippet()`
Trigramas (`pg_trgm`) no título	`LIKE` no `title_sort` sem acentos
Redis Streams, ZSET de atraso e pub/sub	Fila em memória com `ExecutorService` e `ScheduledExecutorService`; eventos por Server-Sent Events; histórico em `jobs`
`SELECT ... FOR UPDATE SKIP LOCKED`	`BEGIN IMMEDIATE` e uma conexão só, protegida por trava
`pg_dump` e cópia do volume	`VACUUM INTO` + zip da pasta `documentos/` (Ferramentas)
ORM e migrações do Django	`JdbcTemplate` do Spring, lista `Migrations.ALL` e `PRAGMA user_version`

CAPÍTULO 12

Histórico de versões

O que mudou no banco a cada versão do Lume para Windows.

VERSÃO	ESQUEMA	MUDANÇAS
2.1.0	`user_version 2`	Sem migração. Novas chaves `watch_folders` e `webhooks` em `settings`; novos usos de `source_path` (pastas monitoradas) e de `finished_at` (Estatísticas e evento `shelf.finished`); backup com `VACUUM INTO`; manutenção pela tela Ferramentas; consultas das Estatísticas e dos Parecidos.
2.0.0	`user_version 2`	Núcleo em Java com `sqlite-jdbc` e `JdbcTemplate`; mesmo esquema da versão em Electron, aberto sem conversão; chave de IA passa a usar o prefixo `dpapi:`.
1.x	1 → 2	Esquema inicial; a migração 2 converte as cores das anotações para a paleta (Sálvia, Rosé, Ardósia, Camurça).

Lume para Windows · Banco de dados · versão 2.1.0 · outubro de 2026. Copyright © 2026 Erik Pimenta e Clara Portes. Distribuído sob a licença MIT. Composto em Literata, Public Sans e IBM Plex Mono a partir de `docs/fonte/banco-de-dados.html`; o diagrama e as fichas foram conferidos contra `Migrations.java`.